ENGRAM//KIT

educational manga reading pack

The Notebook That

Redesigns Itself · ALMA

github.com/romyilano/engram-kit

Watch.
Engy will solve this little maze.

Done!
That was easy.

But a model with no memory is *stateless* — afterwards, the notebook empties.

...Have I seen this before?
?

Why does it feel so hard again?

With no memory, every task starts from zero.
You can't keep what you can't remember.

So we give Engy a memory module — a place to keep and reuse what it learns.

MEMORY MODULE

Now the note survives. Stored experience can be reused.

Maze rule: Follow the open path. Look ahead.

Wait — I've done something like this before...

Maze rule: Follow the open path.

General idea: Find the path that leads to the goal.

Faster this time! Memory helps.

It does. But look closely at the box itself...

MAZE

PUZZLES

PATHS

RULES

Who designed these slots — what to keep, where to file it, what to forget?

?

MAZE

PUZZLES

PATHS

RULES

FACTS

OBSERVATIONS

TRICKS

OTHER

FIXED STRUCTURE

For years, we drew the memory design by hand — what to store, how to fetch it, when to update.

MEMORY DESIGN (HAND-CRAFTED)

STORE (What)
- Step
- Result
- Context

RETRIEVE (How)
- Match
- Rank
- Return

UPDATE (When)
- Add
- Revise
- Prune

MADE FOR THIS TASK:

KITCHEN CHORES.
- Recipes
- Ingredients
- Steps

Hand-crafted for one task, it fits beautifully.

RECIPE: SOUP
- Carrot
- Onion
- Potato
- Water

STEPS
1. Chop
2. Boil
3. Simmer
4. Serve

Soup recipe Success!

KITCHEN

This world is... different. Nothing lines up.

?

SAME DESIGN APPLIED HERE ↓

TILE PUZZLE WORLD

RETRIEVE (How)
- Match
- Rank
- Return

UPDATE (When)
- Add
- Revise
- Prune

A fixed design can't adapt to new, changing tasks.

??

Diverse, shifting tasks each want a different memory. One hand-drawn shape can't serve them all.

MANY WORLDS

KITCHEN

WORD PUZZLE

TILE GAME

DUNGEON

... AND MORE

ONE FIXED DESIGN

STORE → RETRIEVE → UPDATE ✗

So... what if the design didn't have to be drawn by hand?

KITCHEN WORLD

WORD PUZZLE WORLD

TILE GAME WORLD

DUNGEON WORLD

DIFFERENT WORLDS. DIFFERENT NEEDS. DIFFERENT MEMORY SHAPES.

ALMA *doesn't do the task. It designs the memory — over and over.*

1 · SAMPLE

THE ARCHIVE

Design #042
Retrieval: Recency
Update: Append
0.3

2 · REFLECT

This old design only scored 0.3. Why?

Design #042
Retrieval: Recency
Update: Append
Success: 0.3
0.3
EVAL LOG
1.0
0.5
0.0

3 · IDEATE & PLAN

What if we change how it retrieves and updates?

PLAN
New idea:
- Hybrid retrieval
- Relevance + Recency
- Summarize on update

4 · IMPLEMENT (in code)

memory_design_v2.py

```
class MemoryDesignV2:
  def retrieve(query, memory):
    # Hybrid: relevance + recency
    scores = relevarice(query, memory)
    scores += recency(memory)
    return topk(scores, k=5)

  def update(event, memory):
    # Summarize + append
    summary = summarize(event)
    memory.append(summary)
    prune_if_needed(memory.)
```

Design v2
Hybrid Retrieval
↓
Update: Summarize + Append

5 · VERIFY

Syntax OK. All tests pass.

RUN CHECKS
✓ Parse code
✓ Unit tests
✓ Simulated ops
✓ Interface valid
RESULT: ✓ PASS

Design v2

6 · EVALUATE on tasks

Try it on a real task: 0.3 → 0.4.

Install
Design v2

EVALUATION LOG
Old (v1)
Success: 0.3
0.3
→
New (v2)
Success: 0.4
0.4

7 · STORE → repeat

Keep the result. Sample again. This open-ended loop never stops improving.

NEW!
Design #117
Retrieval: Hybrid (Relevance+Recency)
Update: Summarize + Append
Success: 0.4
0.4

ALMA's Meta Agent *meta-learns the memory design through an open-ended loop.*

A memory design is just... code.

```
class MemoryDesign:
  Schema = {
    entities: [Note, Link, Fact],
    fields: {
      Note: [id, text, tags, time],
      Link: [src, dst, rel, time],
      Fact: [subj, pred, obj, conf, time]
    }
  }

def retrieve(query, state):
  qv = embed(query)
  cands = graph_expand(state, qv, hops=2)
  scored = rank(cands, qv)
  return topk(scored, k=5)

update(state, obs):
  new = extract(obs)
  state = merge(state, new)
  state = decay(state)
  return state
```

MEMORY DESIGN (SCHEMATIC)

What we store, how we fetch it, how it changes.

(a) SCHEMA

the shape of what's stored

Note { id, text, tags, time }

Link { src, dst, rel, time }

Fact { subj, pred, obj, conf, time }

(b) RETRIEVAL

how a memory is fetched

Query

Embed

Graph Expand (hops=2)

Rank

Top-k Results

(c) UPDATE

how memory changes over time

New Observation

Extract

Merge

Decay / Prune

New State

A list? A database? A graph? Code can build any of them.

LIST DESIGN

```
[
  Note 1
  Note 2
  Note 3
  ...
  Note N
]
```

DATABASE DESIGN

Notes

id	text	tags	time
1	...	...	...
2	...	...	...

Links

src	dst	rel	time
7	3	cites	...
∴	...	...	...

Facts

subj	pred	obj	conf	time
cat	likes	fish	0.9	...
...	...	...	...	...

GRAPH DESIGN

Different shapes. Different strengths. All expressible in code.

Install the new design straight into Engy's memory.

NEW DESIGN (Graph + Time Decay)

```
retrieve = graph_expand(...)
update = merge(...)
decay = time_weighted(...)
```

This shape suits this task!

TASK:

RETRIEVE

Query

Top-k

UPDATE

New Obs

Merge + Decay

New State

RESULT:

And the space of possible memories is endless.

```
class MemoryDesign:
  ...
```

Countless designs. Infinite possibilities.

1 Every design ever tried is kept here — with its score.

THE ARCHIVE
Memory Designs & Evaluation Logs

2 0.2... 0.4... 0.7 — the archive remembers what worked.

0.2

0.4

0.7

3 Start from a design that already did well.

0.73

```
class MemoryDesign:
  schema = Graph()
  retrieve = topk_sim()
  update = merge()
  decay = time_weighted()
  index = hnsw()
  ...
```

4 ...then make it better.

Parent Design (score: 0.73)

```
retrieve = topk_sim()
update = merge()
decay = time_weighted()
index = hnsw()
...
```

New Design (v2)

Better Design!

```
retrieve = graph_expand()
update = merge+decay()
decay = learned_rate()
index = hnsw_mixed()
prune = utility_weighted()
...
```

5 File the result back. Good designs seed even better ones.

NEW!

0.86

6 This is open-ended search — it compounds. It never has to stop.

Better ideas spread.
The search compounds over time.

1 Four very different worlds:
ALFWorld · TextWorld · Baba Is AI · MiniHack.

ALFWorld
(household chores)

TextWorld
(worlds made of words)

> look around
You are in a library.
Exits: north, east.
> go north
You see a key
> take key
Taken.

Baba Is AI
(rule-pushing puzzle)

BABA BABA IS YOU
WALL IS STOP

MiniHack
(dungeon crawl)

HP: 12/12
Gold: 23
Str: 8 Lev: 3

2 Each world,
its own memory design.

ALFWorld
Design A

TextWorld
Design B

Baba Is AI
Design C

A IS B
B IS C
C IS D

MiniHack
Design D

3 I'm actually learning from experience now!

Wipe Table ✓
Wash Dishes ✓
Put Mug Away ✓
Water Plant ✓

TextWorld
> open door
You unlocked it.
> go north
You find a treasure chest!
You win!

Baba Is AI

BABA BABA IS YOU
WALL IS STOP
FLAG AGAIN IS WIN

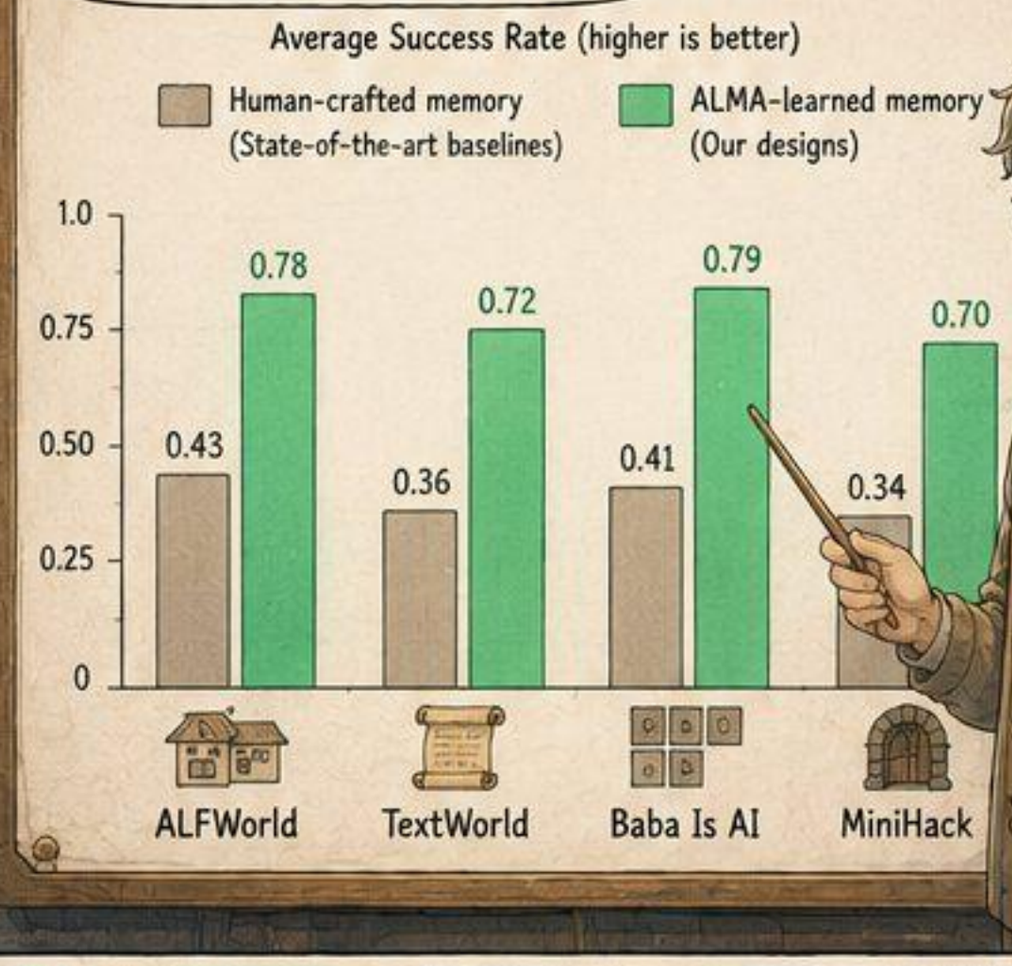
MiniHack
You hit the goblin.
Goblin defeated!
You find a potion.
Level up!
HP: 16/16 Lev: 4

4 Across every benchmark, the learned designs beat the hand-crafted ones.

Average Success Rate (higher is better)

Human-crafted memory
(State-of-the-art baselines)

ALMA-learned memory
(Our designs)

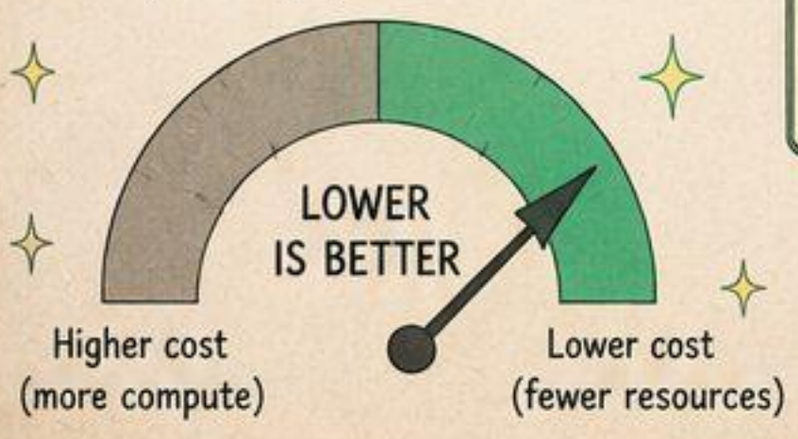


5 And they cost less to run.

Average Compute Cost per Episode
(lower is better)

Human-crafted memory
(State-of-the-art baselines)

ALMA-learned memory
(Our designs)



RESULT

Better performance with fewer resources.

6 Better memory — discovered, not hand-drawn.

THE RESULTS

- Consistently better ✓
- Across 4 diverse worlds ✓
- More cost-efficient ✓

LEARNED TO REMEMBER,
REMEMBERED TO LEARN.

PANEL 1

PANEL 2

PANEL 3

Memory That Designs Itself

panel art to be generated in ChatGPT and dropped in here